EXECUTIVE REPORT

Connected Government Services: Using AI to Help Applications Work Together

A practical assessment of how AI-enabled applications could coordinate across public-sector service boundaries, with a working SDPR + Water Permits + BC Laws proof of concept.

Executive Summary

We tested whether separate government applications could work together to answer a broader citizen-service question. The conclusion is not that one AI tool should be forced onto every application. The more useful conclusion is that AI can help coordinate independently operated systems, if those systems expose clear, safe, well-governed interfaces.

In this model, the important unit is the application, not the AI model. Each application has its own data, rules, security boundary, and accountable owner. AI can help route questions to the right application and combine the responses, but it should not hide who owns the answer or who is responsible for the decision.

> **Core finding:** The value is not in the acronym or framework. The value is in making applications discoverable, callable, auditable, and able to collaborate safely across program boundaries.

The strategic opportunity for BC Gov is not simply "build more chatbots." The opportunity is to make service systems discoverable, interoperable, policy-aware, and easier to coordinate across program boundaries.

What We Tested

The experiment connected three service areas and one real OCR/API boundary:

SDPR form review

Accepts OCR-derived values from an SDPR Monthly Report and prepares benefit-validation notes. It does not make eligibility or payment decisions.

Synthetic workflow | Real OCR values

Document Intelligence OCR layer

The local coordinator uploads a form image to the hosted BC Gov Document Intelligence API, which runs an Azure Document Intelligence workflow behind the scenes.

Hosted API | Azure-backed OCR

Water permit triage

Handles a separate water-permit question about a temporary pump withdrawal near a stream. It prepares triage notes only.

Synthetic workflow | Officer review boundary

BC Laws reference service

Uses a revived BC Laws Neo4j-backed retrieval service as a shared legislation/reference layer for both SDPR and Water workflows.

Neo4j retrieval | Reference only

This was intentionally a cross-domain scenario. A person submitting SDPR information may also have a related question about a change of address, worksite, permit, Service BC process, FOI, identity, or another public service. Today, those questions often live in separate systems and separate teams. Connected application workflows are one possible pattern for reducing that fragmentation.

Demo Workflow

The demo prompt was:

```
An SDPR worker is reviewing the uploaded Monthly Report.
Use the OCR values from the form for benefit validation.
The same client also says their address/worksite changed and asks whether
a temporary pump withdrawal from a nearby stream needs authorization.
Route through BC Laws as the shared reference layer before producing SDPR
and water-permit triage notes.
upload /home/antshiv/Downloads/sdpr_demo_form_1.jpeg
```

Observed workflow

1. The coordinator accepted the user prompt and local form path.
2. The local coordinator uploaded the form to the hosted BC Gov Document Intelligence API.
3. The Document Intelligence API ran the Azure-backed OCR workflow.
4. The coordinator waited for completion and received extracted OCR values.
5. The SDPR service used the OCR values to prepare worker-review notes.
6. The SDPR and Water services consulted the BC Laws reference service.
7. The Water service prepared permit triage notes.
8. The coordinator returned one combined answer showing each specialist boundary.

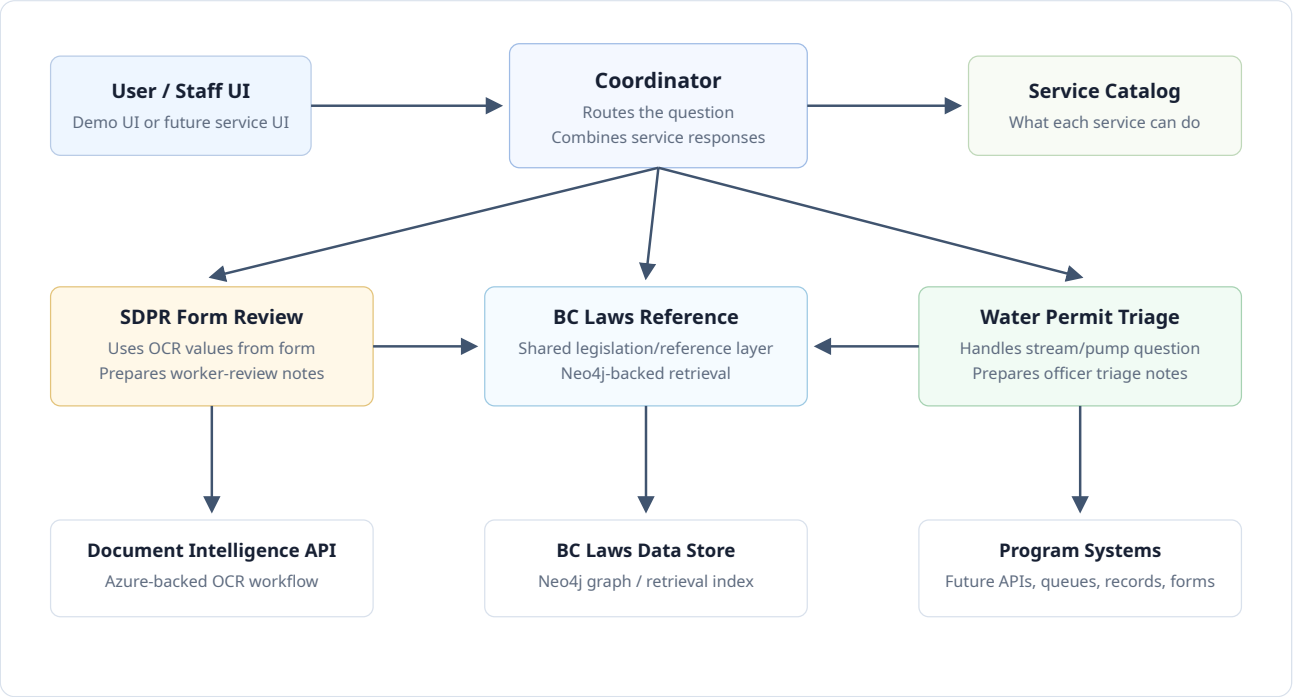
Example OCR-derived SDPR facts

Field	Extracted value	Use in demo
Applicant	Patrick O'Connor	Displayed in the SDPR worker-review package.
Spouse	Lisa O'Connor	Displayed in the SDPR worker-review package.
Employment Insurance	$343.25	Included in candidate reportable income.
Room / Board Income	$333.85	Included in candidate reportable income.
OAS / GIS	$1,580.23	Included in candidate reportable income.
Income Tax Refund	$26.80	Included in candidate reportable income.
Change note	changed address, proof of move	Triggers address/shelter review note.

The SDPR and Water workflows in this proof of concept are simulated. The Document Intelligence API call and BC Laws graph-backed retrieval are real integration paths in the demo environment. No production eligibility, payment, permit, or legal decision is made.

Architecture

The important architecture principle is that each application keeps its boundary. The coordinator does not need to know every database, model, prompt, or internal tool. It needs to know what each service can safely do, how to call it, and what trust/risk rules apply.



How the pieces fit together

Piece	Plain-language role	Example in this demo
Service/application	The actual program capability, data, workflow, or API.	Document Intelligence API, BC Laws graph, SDPR validation logic, Water triage logic.
Service interface	A controlled way for one application to describe what it can safely do for another application.	SDPRValidationSpecialist, WaterPermitSpecialist, BCLawsReferenceSpecialist.
Application-to-application communication	A way for independently operated services to discover and call each other.	Each specialist exposes a small HTTP endpoint and a short description of what it can do.
Tool/data access	A way for a service to access approved tools, APIs, databases, files, or other systems.	Could expose Neo4j, search, records, or OCR tools to an approved service.
Coordinator	Routes a user task to the right services and composes the	Google ADK in the local demo; Microsoft Agent Framework and WayFlow were also

Piece	Plain-language role	Example in this demo
	response.	explored.

Technical Appendix: Tools and Protocols

The report intentionally avoids leading with technical acronyms. For implementation teams, these are the tools and protocols explored:

Tool / protocol	What it is	How to think about it
Google ADK	A framework for building, testing, and deploying AI-enabled applications. It includes a local developer UI.	The coordinator used in the working demo.
Microsoft Agent Framework	Microsoft's framework for building and coordinating AI-enabled application workflows.	Another coordinator option, especially relevant in Microsoft-heavy environments.
Oracle Agent Spec	A specification for representing AI-enabled services and flows.	A portability/specification layer; it does not replace ADK or Microsoft by itself.
Oracle WayFlow	A reference runtime for Oracle Agent Spec.	A way to execute Agent Spec-style flows.
A2A	A protocol pattern for application-to-application AI interoperability.	How one service can discover and call another service.
MCP	A protocol for exposing tools/context to agents.	How an AI-enabled service can use tools such as databases, APIs, files, search, or internal services.

> **Correction to avoid overclaiming:** Oracle Agent Spec does not sit "above" Google ADK or Microsoft Agent Framework. It defines a specification. WayFlow is one runtime for that specification. ADK and Microsoft Agent Framework can still orchestrate agents directly, especially when those agents expose compatible A2A/API surfaces.

Relationship to coding assistants

Tools such as Codex, Claude Code, Gemini CLI, and GitHub Copilot are primarily developer assistants. They can help build or operate these systems, but they are not the same thing as a governed service coordination layer. ADK and similar frameworks are about building AI-enabled applications and coordinating them across system boundaries.

Governance and Architecture Implications

For this pattern to work in BC Gov, there must be an architecture approach for applications that expose AI-enabled or API-enabled capabilities to other applications. Without that, every team will build a disconnected assistant, and citizens will still have to navigate the organization manually.

Recommended direction

- Define what it means for a BC Gov application to safely expose a service interface for other applications.
- Separate service ownership from coordination ownership.
- Require clear service descriptions: capability, owner, data classification, allowed actions, authentication, and decision boundary.
- Use BC Laws and policy/reference systems as shared grounding layers where appropriate.
- Support multiple integration styles where useful: plain HTTP APIs, service-to-service protocols, tool connectors, queues, and existing integration platforms.
- Design for audit, observability, consent, privacy, and least-privilege access from the start.

Risks

Security and privacy

Connected-service communication can increase data movement. Every call must respect classification, consent, authorization, and audit requirements.

Unnecessary complexity

Not every application needs an AI interface. Some systems only need a clean API, a search endpoint, or better documentation.

False authority

A grounded answer is not automatically a legal, eligibility, payment, or permit decision. Decision boundaries must be explicit.

Operational ownership

Backbone services such as BC Laws retrieval need a mandate, support model, monitoring, data-refresh process, and product owner.

Why BC Laws matters

BC Laws is not just another demo data source. It can become a shared reference backbone for

multiple services if it is kept current, versioned, searchable, and exposed safely. In the proof of concept, both SDPR and Water workflows used BC Laws to identify candidate reference areas. That pattern could apply to many services listed under BC Gov Services A-Z.

> **Important boundary:** BC Laws grounding should identify candidate source areas for review. It should not be presented as legal advice or final statutory interpretation unless an approved legal process owns that output.

Potential Benefits

- **Citizen experience:** A citizen can ask one service question that spans multiple program areas instead of discovering the government structure themselves.
- **Reuse:** Existing applications can expose focused capabilities rather than rebuilding a new full application for every use case.
- **Grounding:** Answers can be connected to current laws, policies, forms, and program rules.
- **Operational clarity:** Each application keeps its owner, boundary, data access, and accountability.
- **Modernization path:** Legacy systems can participate through APIs, wrappers, queues, or adapters without being immediately replaced.

What This Does Not Prove

- It does not prove production readiness.
- It does not prove that SDPR, Water, BC Laws, or any other real program should adopt this exact implementation.
- It does not remove the need for privacy, security, legal, records-management, and architecture review.
- It does not mean every system should become an AI-enabled service.
- It does not mean the coordinator should own all business logic.

The proof of concept demonstrates an integration pattern: a coordinator can route a complex request to independent service boundaries, gather outputs, and return a more comprehensive answer while keeping each specialist's responsibility visible.

Recommended Next Steps

1. Create a short connected-application interoperability standard for BC Gov experiments.
2. Define required metadata for any exposed service endpoint: owner, purpose, allowed actions, auth method, data classification, audit model, and decision boundary.
3. Revive BC Laws as a properly governed backbone experiment, not a one-off demo.
4. Test the pattern with two or three friendly program areas using synthetic or approved non-sensitive data.
5. Compare candidate technical frameworks on deployment, governance, observability, interoperability, and integration fit.
6. Document where a plain API is enough and where an AI-enabled interface adds value.

References

- Google Agent Development Kit documentation
- Google ADK A2A quickstart
- Google ADK MCP tools documentation
- Microsoft Agent Framework orchestration documentation
- Oracle WayFlow GitHub repository
- Oracle Agent Spec GitHub repository
- BC Gov Services A-Z